

MEMORIA PRÁCTICA 1 - PROCESADORES DE LENGUAJE

Enumeración y especificación formal de definiciones auxiliares en Tiny(0) y Tiny

Expresiones regulares elementales para construir definiciones de nuestras clases léxicas.

Nombre	Descripción	Expresión Regular
letra	Carácter dentro del alfabeto inglés.	$[a-z, A-Z]$
dígito	Carácter numérico.	$[0, 9]$
natural	Carácter numérico positivo.	$[1, 9]$
entero	Parte entera de un número.	$\{natural\}\{dígito\}^* \mid 0$
decimal	Parte decimal de un número.	$\{dígito\}^*\{natural\} \mid 0$
exponencial	Parte exponencial de un número	$(e E)(+ - \epsilon)\{entero\}$

Enumeración de clases léxicas en Tiny(0)

Empezamos definiendo ciertos términos para su futuro uso en la especificación. Tiny(0) es un subconjunto de Tiny que incluye los siguientes elementos básicos que pueden ser reconocidos por el analizador léxico dependiendo de sus características:

Clases léxicas univaluadas:

Nombre	Descripción
Palabras reservadas	<i>int</i> <i>real</i> <i>bool</i> <i>and</i> <i>or</i> <i>not</i> <i>true</i> <i>false</i>
Operadores aritméticos	<i>sum</i> <i>rest</i> <i>mul</i> <i>div</i> <i>mod</i>
Operador lógico	<i>and</i>

	or not
Operador relacional/asignación	menor mayor menorIgual mayorIgual igual diferente asignación
Operador de puntuación	parentesisApertura parentesisCierre corcheteApertura corcheteCierre llaveApertura llaveCierre coma punto and andDoble arroba puntoYComa
Cadena ignorable	Espacios en blanco y comentarios ##

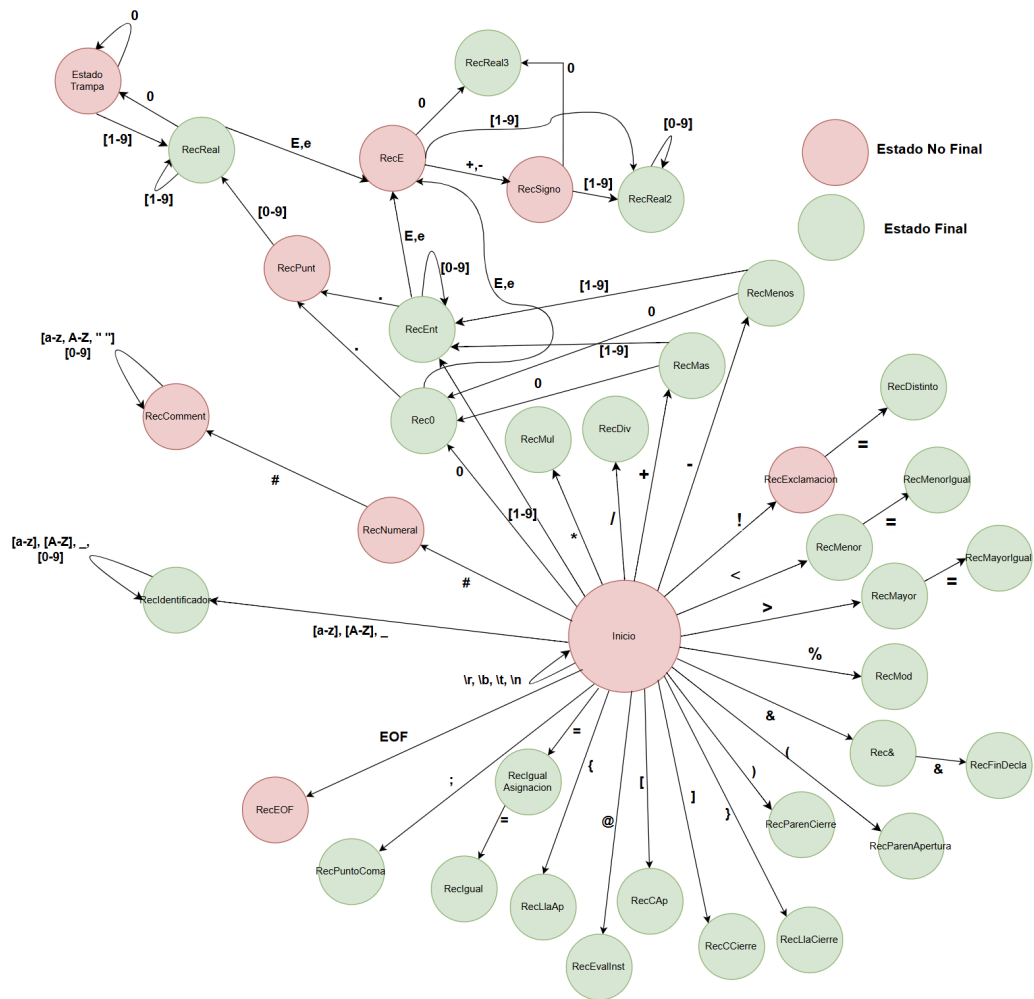
Clases léxicas multivaluadas:

Nombre	Descripción
Identificadores	Comienzan con una letra (a-z, A-Z) o “_”, puede contener letras, dígitos (0-9) o “_”. Son sensibles a mayúsculas
Literales Enteros	Pueden opcionalmente iniciar con un + o -, no permiten ceros no significativos.
Literales reales	Comienzan obligatoriamente con una parte entera, seguida de una parte decimal o bien de una parte exponencial. La decimal comienza con un punto seguida de una secuencia de 1 o más dígitos sin incluir ceros no significativos
Literal Booleanos	true o false

Basándonos en las definiciones anteriores podemos construir las siguientes expresiones regulares en Tiny(0):

Nombre	Expresión Regular
Palabras reservadas	<i>tipo int</i> = (i I)(n N)(t T) <i>tipo real</i> = (r R)(e E)(a A)(l L) <i>tipo bool</i> = (b B)(o O)(o O)(l L) <i>and</i> = (a A)(n N)(d D) <i>or</i> = (o O)(r R) <i>not</i> = (n N)(o O)(t T) <i>true</i> = (t T)(r R)(u U)(e E) <i>false</i> = (f F)(a A)(l L)(s S)(e E)
Operadores aritméticos y símbolos	<i>sum</i> = + <i>rest</i> = - <i>mul</i> = * <i>div</i> = / <i>mod</i> = % <i>menor</i> = < <i>mayor</i> = > <i>menorIgual</i> = <= <i>mayorIgual</i> = >= <i>igual</i> = == <i>diferente</i> = != <i>asignación</i> = = <i>parentesisApertura</i> = (<i>parentesisCierre</i> =) <i>corcheteApertura</i> = [<i>corcheteCierre</i> =] <i>llaveApertura</i> = { <i>llaveCierre</i> = } <i>coma</i> = , <i>punto</i> = . <i>and</i> = & <i>andDoble</i> = && <i>arroba</i> = @ <i>puntoYComa</i> = ;
Identificadores	(<i>{letra}</i> <i>_</i>) (<i>{letra}</i> <i>{dígito}</i> <i>_</i>)*
Literal Entero	(+ -)? <i>{entero}</i>
Literal Real	(<i>{entero}</i> . <i>{decimal}</i>) (<i>{entero}</i> <i>{exponencial}</i>) (<i>{entero}</i> . <i>{decimal}</i> <i>{exponencial}</i>)
Espacio Blanco	[\t \r \b \n]
Comentarios	##[<i>^</i> \n]*

Diagrama de transiciones de Tiny(0):



Enumeración de clases léxicas en Tiny

Clases léxicas univaluadas:

Nombre	Descripción
Palabras reservadas	int real bool and or not true false null if else while struct

	new, delete read write nl type call proc
Operadores aritméticos	<i>sum</i> <i>rest</i> <i>mul</i> <i>div</i> <i>mod</i> <i>exp</i>
Operador lógico	and or not
Operador relacional/asignación	<i>menor</i> <i>mayor</i> <i>menorIgual</i> <i>mayorIgual</i> <i>igual</i> <i>diferente</i> <i>asignación</i>
Operador de puntuación	<i>parentesisApertura</i> <i>parentesisCierre</i> <i>corcheteApertura</i> <i>corcheteCierre</i> <i>llaveApertura</i> <i>llaveCierre</i> <i>coma</i> <i>punto</i> <i>and</i> <i>andDoble</i> <i>arroba</i> <i>puntoYComa</i>
Cadena ignorable	Espacios en blanco y comentarios

Clases léxicas multivaluadas:

Nombre	Descripción
Identificadores	Comienzan con una letra (a-z, A-Z) o “_”, puede contener letras, dígitos (0-9) o “_”. Son sensibles a mayúsculas
Literales Enteros	Pueden opcionalmente iniciar con un + o -, no

permiten ceros no significativos.

Laterales reales	Comienzan obligatoriamente con una parte entera, seguida de una parte decimal o bien de una parte exponencial. La decimal comienza con un punto seguida de una secuencia de 1 o más dígitos sin incluir ceros no significativos
Literal Booleanos	true o false
Literales de cadena	Comienzan y terminan con comilla doble, no permiten saltos de línea y se permiten \n, \t, \r, \b

Enumeración de las cadenas ignorables en Tiny

Nombre	Expresión Regular
separador	<code>[\s, \b, \r, \n, \t]</code>
comentario	<code>##[^\n]*</code>

Por último, podemos construir las siguientes expresiones regulares:

Nombre	Expresión Regular
Palabras reservadas	<i>tipo int = (i I)(n N)(t T)</i> <i>tipo real = (r R)(e E)(a A)(l L)</i> <i>tipo bool = (b B)(o O)(o O)(l L)</i> <i>and = (a A)(n N)(d D)</i> <i>or = (o O)(r R)</i> <i>not = (n N)(o O)(t T)</i> <i>true = (t T)(r R)(u U)(e E)</i> <i>false = (f F)(a A)(l L)(s S)(e E)</i> <i>null = (n N)(u U)(l L)(l L)</i> <i>if = (i I)(f F)</i> <i>else = (e E)(l L)(s S)(e E)</i> <i>while = (w W)(h H)(i I)(l L)(e E)</i> <i>struct = (s S)(t T)(r R)(u U)(c C)(t T)</i> <i>new = (n N)(e E)(w W)</i> <i>delete = (d D)(e E)(l L)(e E)(t T)(e E)</i> <i>read = (r R)(e E)(a A)(d D)</i> <i>write = (w W)(r R)(i I)(t T)(e E)</i> <i>nl = (n N)(l L)</i> <i>type = (t T)(y Y)(p P)(e E)</i> <i>call = (c C)(a A)(l L)(l L)</i>

Operadores aritméticos y
símbolos

sum = +
rest = -
mul = *
div = /
mod = %
exp = ^
menor = <
mayor = >
menorIgual = <=
mayorIgual = >=
igual ==
diferente !=
asignación = =
parentesisApertura = (
parentesisCierre =)
corcheteApertura = [
corcheteCierre =]
llaveApertura = {
llaveCierre = }
coma = ,
punto = .
and = &
andDoble = &&
arroba = @
puntoYComa = ;

Identificadores

$(\{letra\} | _)(\{letra\} | \{dígito\} | _)^*$

Literal Entero

$[\backslash + \backslash -]? \{entero\}$

Literal Real

$(\{entero\}.\{decimal\}) | (\{entero\} \{exponencial\}) |$
 $(\{entero\}.\{decimal\} \{exponencial\})$

Literal Cadena

$"([\backslash \^ \backslash "])^*"$

Espacio Blanco

$[\backslash t \backslash r \backslash b \backslash n]$